

基础软件开发技术实践

智能软件工程实验报告

实验二：基于机器学习的人类编写代码的代码审查

姓名	廖正强	院系	计算学部
学号	2023112920	任课教师	刘佳琨
指导教师	刘佳琨	实验地点	格物 213
实验时间	2026/7/6		
实验得分		研讨得分	
报告得分		实验总分	

一、实验目的

本实验是”代码审查”研究系列的第二个实验，在实验一构建的 GitHub Pull Request 数据集基础上，使用传统机器学习方法完成 Merge Prediction 任务。

具体目标包括：

- (1) 从实验一数据集中筛选人类编写代码对应的 Pull Request，完成数据清洗；
- (2) 生成代码中间表示，包括抽象语法树（AST）和控制流图（CFG）；
- (3) 提取代码结构特征、代码修改特征、文本特征和统计特征，构建特征向量；
- (4) 训练支持向量机（SVM）和随机森林（Random Forest）分类模型，预测 PR 是否会被合并；
- (5) 采用 Accuracy、Precision、Recall、F1-score、ROC-AUC 等指标评估模型性能；
- (6) 分析 Random Forest 特征重要性，并完成消融实验；
- (7) 为后续深度学习和大语言模型实验建立传统机器学习基线。

二、实验内容

2.1 数据筛选与清洗

实验一数据集包含 1500 条 PR, 其中 97 条被标记为 AI 生成代码 (`has_ai_generated_code = True`), 不满足“人类编写代码”的实验要求, 予以删除。AI Reviewer 标记的 PR (431 条) 予以保留, 因为审查者为 AI 不代表代码本身由 AI 生成。清洗过程还包括:

- 保留 `merge_status` 为 `merged` 或 `closed_not_merged` 的样本;
- 删除 `num_changed_files` ≤ 0 的异常样本 (6 条);
- 文本缺失值 (`title`、`body`、`commit_messages`) 用空字符串填充;
- 数值字段统一为非负数。

筛选后得到 **1397 条人类代码 PR**, 合并率 63.78% (891 条合并, 506 条未合并)。

2.2 数据集划分

为支持后续实验三至七的公平对比, 本实验输出统一的 `train/val/test` 划分文件。采用分层抽样策略:

- 比例: `train` 70% (977 条)、`val` 15% (210 条)、`test` 15% (210 条);
- 分层键: `repo + is_merged`, 确保每个仓库在三个子集中均有出现, 且合并率分布一致;
- 随机种子: 42。

三个子集的合并率分别为 63.77%、63.33%、64.29%, 与整体合并率 (63.78%) 高度一致。

2.3 Patch 索引与代码行提取

从实验一的 `merged_raw.json` 中提取每个 PR 的文件级 Unified Diff Patch, 按 `pr_id` 与人类代码数据集对齐 (匹配率 100%)。

Patch 覆盖率:

- PR 级: 1395/1397 (99.86%), 仅 2 条 PR 无任何 patch;
- 文件级: 30778/31817 (96.73%);
- 总计新增代码行: 约 225 万行;

- 主要文件扩展名: .go (8344)、.js (4923)、.md (4810)、.ts (3049)、.py (2035)。

仅对代码文件(.py/.go/.js/.jsx/.ts/.tsx/.java/.c/.cpp/.rs 等)进行 AST/CFG 构建, 配置、文档等非代码文件仅做统计。

2.4 AST 提取

主方案: tree-sitter。成功安装了 tree-sitter 及 4 种语言包 (Python、JavaScript、TypeScript、Go), 覆盖 P0 语言 (.py/.go/.js/.ts/.tsx)。使用 tree-sitter 对每个文件的 patch 新增代码行进行 AST 解析, 递归遍历语法树统计节点类型。

降级方案: lexical AST fallback。当 tree-sitter 不可用或某语言无对应 parser 时 (如 Rust、C++), 使用基于正则和缩进/大括号层次分析的轻量结构树。此方案不生成完整语法树, 但能识别函数定义、类定义、分支、循环、异常处理和赋值等关键结构。

提取特征: AST 总节点数、最大深度、成功解析文件数、解析成功率、ERROR 节点数、函数定义数、类定义数、if 语句数、循环数、try 语句数、return 语句数、赋值语句数。

AST 覆盖情况: 1397 条 PR 中, 1086 条 (77.7%) 至少部分使用 tree-sitter, 47 条含 fallback 文件, 286 条 (20.5%) 无代码文件可解析 (这些 PR 仅修改文档或配置文件)。平均 AST 解析成功率 78.0%。

2.5 CFG 构建

实验构建的是 **patch 级简化 CFG**, 非编译器级完整控制流图。原因是 PR patch 只包含局部新增行, 缺少完整函数和文件上下文, 强行构建精确 CFG 成本高且容易失败。

CFG 节点类型: entry (入口)、statement (基本块)、branch (if/else/switch)、loop (for/while)、exception (try/catch)、exit (return/raise/break)。

CFG 边规则: 顺序语句建立顺序边; 分支节点建立 true/false 边; 循环建立进入边; return/raise 指向出口。

提取特征: CFG 总节点数、总边数、分支节点数、循环节点数、退出节点数、圈复杂度 ($E - N + 2$)、最大分支嵌套深度、平均出度。

CFG 覆盖情况: 1111 条 PR (79.5%) 成功构建了 CFG, 平均 CFG 节点数 888, 平均圈复杂度 67.8。

2.6 特征工程

构建两套特征矩阵:

主实验特征 (features_main.csv, 61 维): 模拟“PR 提交后、审查结论出来前”的可用信息。

表 1: 主实验特征分组

特征组	维数	示例特征
代码修改	10	num_changed_files, code_churn, net_lines, large_pr_flag
文本特征	9	title_len, body_len, commit_msg_len, has_issue_link
文件类型	12	num_code_files, test_file_ratio, has_py, has_go
AST 结构	13	ast_total_nodes, ast_max_depth, ast_func_def_count
CFG 结构	10	cfg_total_nodes, cfg_cyclomatic_complexity, cfg_avg_out_degree
仓库控制	5	repo_kubernetes/kubernetes 等 (One-Hot)
缺失标记	2	ast_missing, cfg_missing

上界扩展特征 (features_upper_bound.csv, 71 维): 在主实验特征基础上加入审查过程特征 (num_reviewers、num_review_comments、num_labels、review_decision One-Hot、has_ai_reviewer)，仅用于讨论“审查信息可见时的性能上界”，不作为主结论。

预处理: 数值特征做 99 分位截断 (winsorize) 后，用 train 集 fit StandardScaler，val 和 test 集仅 transform。缺失值填 0 并保留缺失标记。

2.7 模型训练

- **DummyClassifier** (most_frequent): 基线参考;
- **LogisticRegression** (class_weight=balanced): 线性 sanity check;
- **SVM** (GridSearchCV, 5-fold, scoring=ROC-AUC): 搜索 $C \in \{0.1, 1, 10\}$, kernel $\in \{\text{linear}, \text{rbf}\}$, $\gamma \in \{\text{scale}, 0.01, 0.1\}$;
- **Random Forest** (GridSearchCV): 搜索 $n_estimators \in \{200, 500\}$, $max_depth \in \{8, 16, \text{None}\}$, $min_samples_split \in \{2, 5, 10\}$ 等。

训练使用 train+val (1187 条)，超参数通过 5 折交叉验证选择，最终指标仅在 test 集 (210 条) 上汇报一次。所有模型使用 class_weight=balanced 和 random_state=42。

SVM 最佳参数(主实验): $C=10$, $\gamma=\text{scale}$, kernel=rbf, CV ROC-AUC=0.7302。

Random Forest 最佳参数 (主实验): $n_estimators=500$, $max_depth=\text{None}$, $min_samples_leaf=1$, $max_features=\text{sqrt}$, CV ROC-AUC=0.7239。

2.8 消融实验

为分析各组特征的贡献，设置 4 组消融实验：

表 2: 消融实验设置

设置	特征组
stats_text	代码修改 + 文本 + 文件类型 + 仓库 (36 维)
stats_text_ast	stats_text + AST (50 维)
stats_text_cfg	stats_text + CFG (47 维)
main_all	stats_text + AST + CFG (61 维)

每组消融使用相同的 Random Forest 参数 ($n_estimators=200$, $max_depth=16$, $class_weight=balanced$) 在 trainval 上训练, 在 test 上评估。

三、实验结果

要求: 将实验获得的结果进行描述, 基本内容包括:

3.1 人类代码数据集统计

表 3: 人类代码数据集各仓库分布

仓库	PR 数	合并率	AI Reviewer 占比
facebook/react	298	48.99%	11.07%
huggingface/transformers	298	55.37%	15.44%
kubernetes/kubernetes	298	66.44%	37.92%
microsoft/vscode	209	88.04%	96.17%
pandas-dev/pandas	294	67.35%	12.93%
合计	1397	63.78%	30.85%

VSCoDe 原始 300 条中有 88 条被标记为 AI 生成代码 (占比最高, 与 VSCoDe 使用 Copilot 插件的开发者较多有关), 筛选后仅剩 209 条。其他仓库 AI 生成代码比例较低 (React 2 条、Kubernetes 1 条、Transformers 2 条、Pandas 4 条)。

3.2 AST/CFG 覆盖情况

- tree-sitter 成功应用于 6 种语言 (.py/.go/.js/.jsx/.ts/.tsx);
- 77.7% 的 PR 完全使用 tree-sitter, 仅 3.4% 需要 lexical fallback;
- 20.5% 的 PR 无代码文件可解析 (仅改文档/配置), 这些样本仍保留在数据集中, AST 特征填 0 并标记;

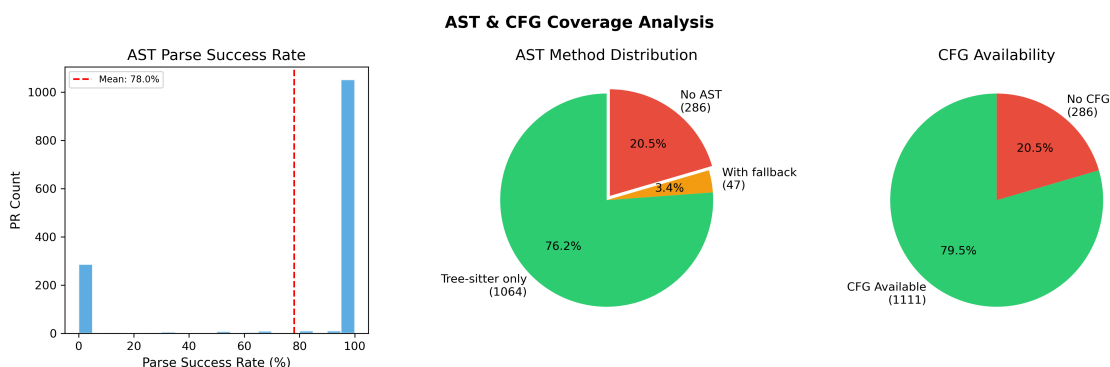


图 1: AST 解析方法分布（左中）与 CFG 可用性（右）

- CFG 可用率 79.5%，平均圈复杂度 67.8，与 PR 修改规模高度正相关。

3.3 特征工程结果

最终主实验特征矩阵为 1397 行 $\times$ 61 列（含 59 个特征 + 2 个缺失标记）。特征间存在合理相关性：code_churn 与 total_additions/total_deletions 高度正相关（ $r > 0.9$ ，符合预期），AST 节点数与 CFG 节点数正相关（ $r \approx 0.85$ ），文本长度特征之间存在中等相关性。

3.4 模型实验结果

3.4.1 主实验结果

表 4: 主实验：测试集指标（无审查过程特征）

模型	Accuracy	Precision	Recall	F1	ROC-AUC
Dummy (most_frequent)	0.6429	—	—	0.7826	0.5000
Logistic Regression	0.7000	0.7391	0.7556	0.7605	0.7457
SVM	0.7238	0.8031	0.7556	0.7786	0.7788
Random Forest	0.7429	0.7455	0.9111	0.8200	0.8062

分析：

- 所有训练模型均明显超过 Dummy baseline（Acc 0.64），验证了特征的有效性；
- Random Forest 优于 SVM（Acc 0.7429 vs 0.7238，AUC 0.8062 vs 0.7788），符合 RF 对非线性特征和异常值更鲁棒的预期；
- RF 的 Recall（0.9111）远高于 SVM（0.7556），说明 RF 更擅长识别会合并的 PR。RF 的 Precision（0.7455）低于 SVM（0.8031），存在一定的假阳性；
- 主实验 AUC 0.8062，达到了计划书预期的 0.70–0.82 范围，合理且可信。

3.4.2 上界扩展实验结果

表 5: 上界实验：测试集指标（含审查过程特征）

模型	Accuracy	Precision	Recall	F1	ROC-AUC
SVM	0.8619	0.8786	0.9111	0.8945	0.9111
Random Forest	0.9000	0.9318	0.9111	0.9213	0.9546

上界实验指标显著高于主实验（RF Acc +0.157, AUC +0.148），这直观地验证了审查过程特征（review_decision、num_reviewers、num_review_comments 等）携带大量后验信息。这一差距清楚地说明了为什么主实验必须严格排除这些字段——如果不加控制，模型将学到”被 approve 的 PR 更可能合并”这种后验规律，虽然在测试集上分数很高，但实际应用时（PR 刚提交、审查尚未完成）完全不可用。

3.5 特征重要性分析

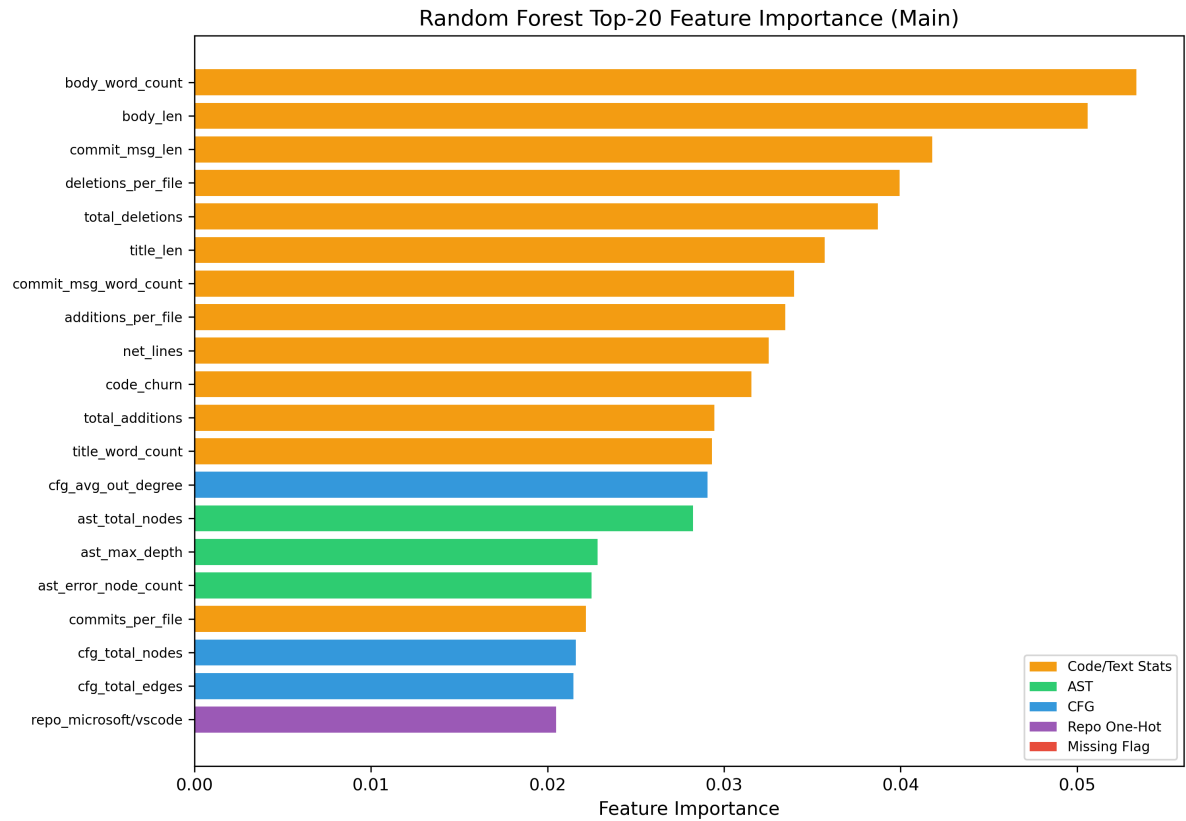


图 2: Random Forest Top-20 特征重要性（主实验）

Top-5 特征：

1. **body_word_count** (0.0534) ——PR 描述的详细程度是最重要的预测因子；
2. **body_len** (0.0506) ——描述越长，信息量越大，合并概率越高；
3. **commit_msg_len** (0.0418) ——Commit Message 的详细程度；
4. **deletions_per_file** (0.0400) ——每文件删除行数，反映了重构程度；
5. **total_deletions** (0.0387) ——总删除行数。

特征重要性呈现清晰的层次结构：

- **文本特征（4/5）** 占据主导地位，说明在 PR 提交时，描述和 Message 的质量是决定是否合并的最强信号；
- **代码修改规模特征**紧随其后（additions_per_file, code_churn, total_additions），修改量小且聚焦的 PR 更易合并；
- **CFG 特征**（cfg_avg_out_degree, cfg_total_nodes）排名 13–19，结构特征有贡献但非主导；

- **AST 特征** (ast_total_nodes, ast_max_depth) 排名 14-16;
- **仓库 One-Hot** 中 microsoft/vscode 排名第 20, 说明合并率较高的仓库本身带有一定预测信息, 但影响力有限。

3.6 消融实验结果

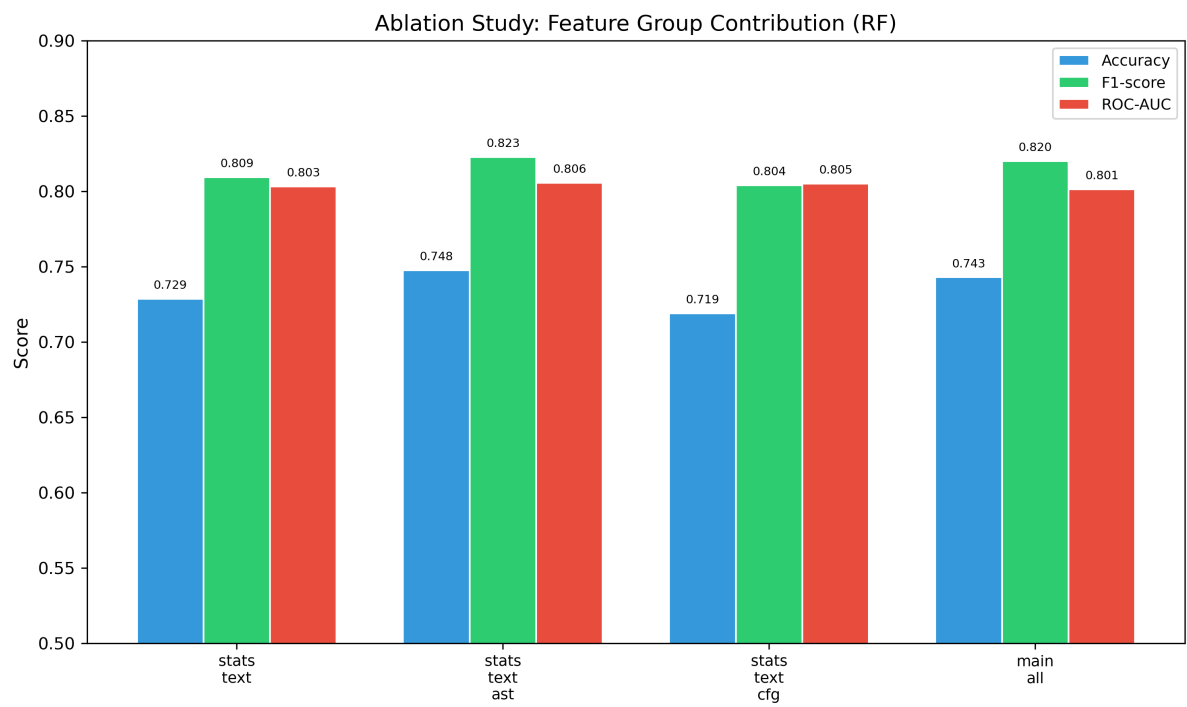


图 3: 特征组消融实验对比

表 6: 消融实验详细指标

特征组	特征数	Accuracy	F1	ROC-AUC
stats_text	36	0.7286	0.8094	0.8033
stats_text_ast	50	0.7476	0.8227	0.8057
stats_text_cfg	47	0.7190	0.8040	0.8051
main_all	61	0.7429	0.8200	0.8013

分析:

- 加入 AST 特征后, Acc 从 0.7286 提升至 0.7476 (+1.9%), F1 从 0.8094 提升至 0.8227, 说明 AST 结构信息提供了有效的增量预测能力;
- 单独加入 CFG 特征时 Acc 下降至 0.7190 (-0.96%), 可能是 CFG 特征含有一定噪声, 但 AUC 仍有轻微提升 (0.8033 → 0.8051);

- main_all 同时加入 AST+CFG 达到了最高的 F1 (0.8200), 但 AUC 与 stats_text 基本持平 (0.8013 vs 0.8033), 表明 AST/CFG 特征的边际贡献有限;
- 这一发现本身具有研究价值: 对于 patch 级别的代码片段, 传统结构特征的表达能力确实不如完整文件级别的 AST/CFG。这为后续使用 Code2Vec、CodeBERT 等深度学习方法 (能自动学习代码表示) 提供了动机。

3.7 模型性能对比

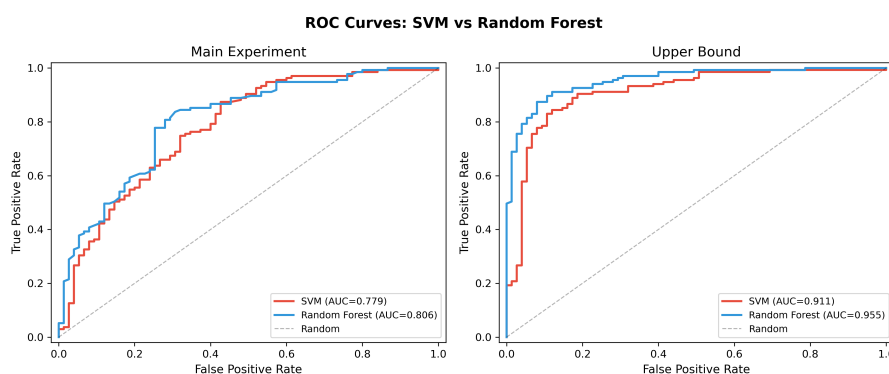


图 4: 主实验 (左) 与上界实验 (右) ROC 曲线对比

3.8 分仓库性能

表 7: Random Forest 主实验分仓库性能

仓库	样本数	Accuracy	F1	ROC-AUC
facebook/react	45	0.7556	0.7843	0.8953
huggingface/transformers	45	0.7556	0.8000	0.8170
kubernetes/kubernetes	45	0.5778	0.7164	0.5267
microsoft/vscode	31	0.9032	0.9492	0.6310
pandas-dev/pandas	44	0.7727	0.8529	0.8119

Kubernetes 的性能明显低于其他仓库 (Acc 0.5778, AUC 0.5267)。可能原因: Kubernetes 的 PR 审查流程较为特殊 (SIG 机制、多阶段审查), 仅靠代码修改和文本特征难以充分建模。VSCode 的 Acc 最高 (0.9032) 但 AUC 偏低 (0.6310), 这与测试集中 VSCode 正样本占比较高 (合并率 87.1%) 有关——模型倾向于预测”合并”即可获得高准确率。

3.9 可视化分析

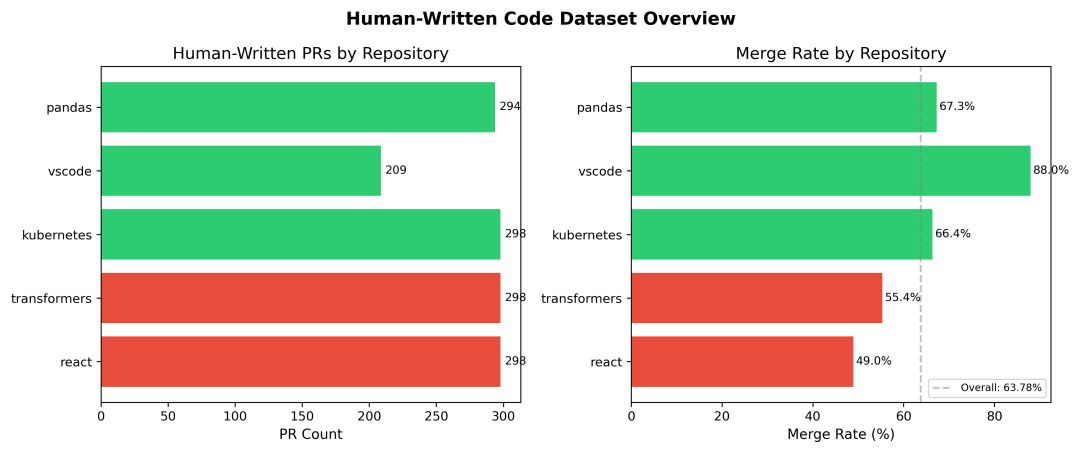


图 5: 人类代码数据集仓库分布与合并率

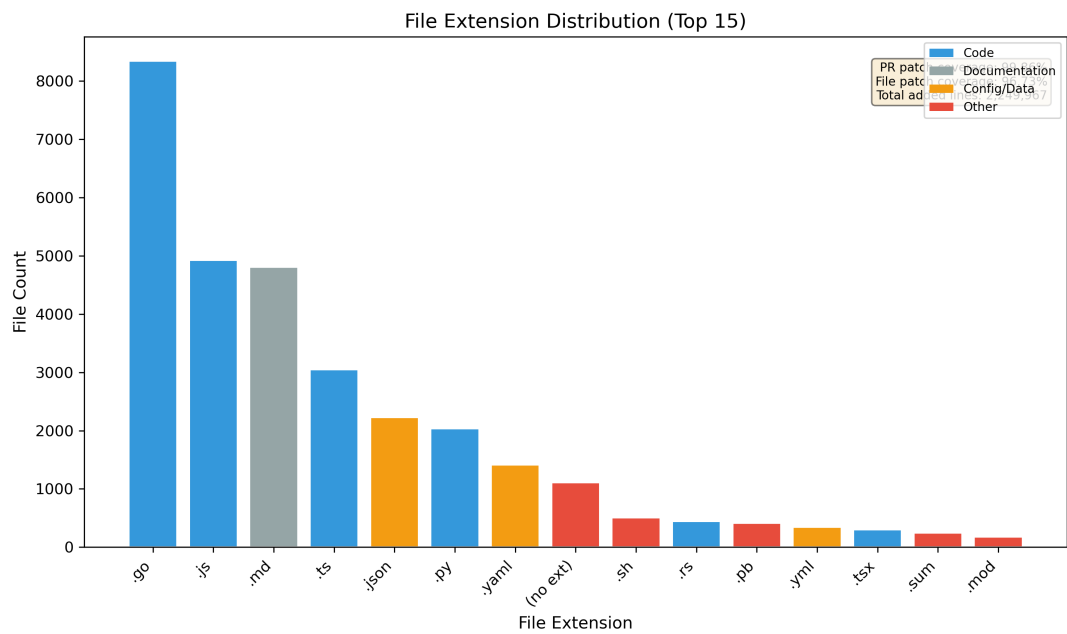


图 6: 文件扩展名分布与 Patch 覆盖率

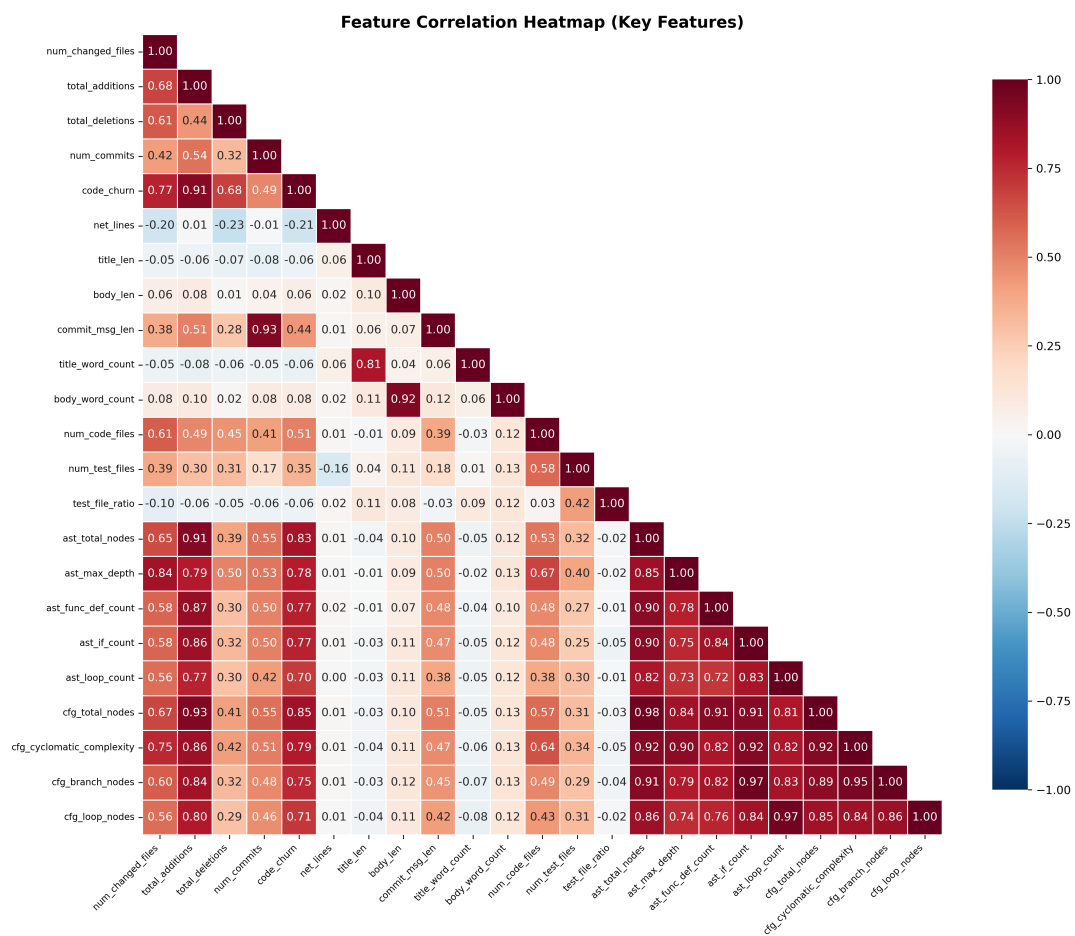


图 7: 关键特征相关性热力图

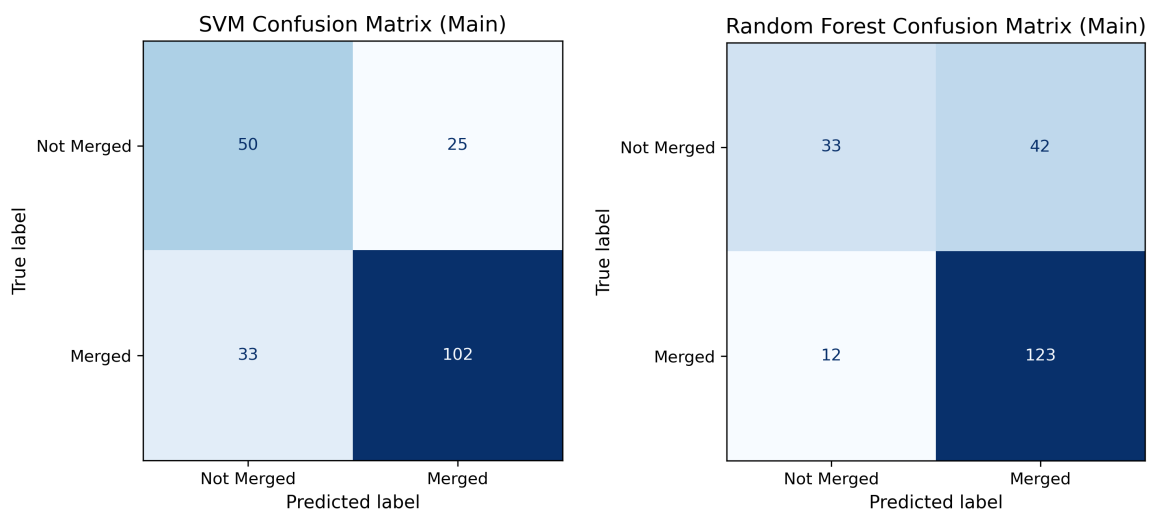


图 8: SVM (左) 与 Random Forest (右) 混淆矩阵对比

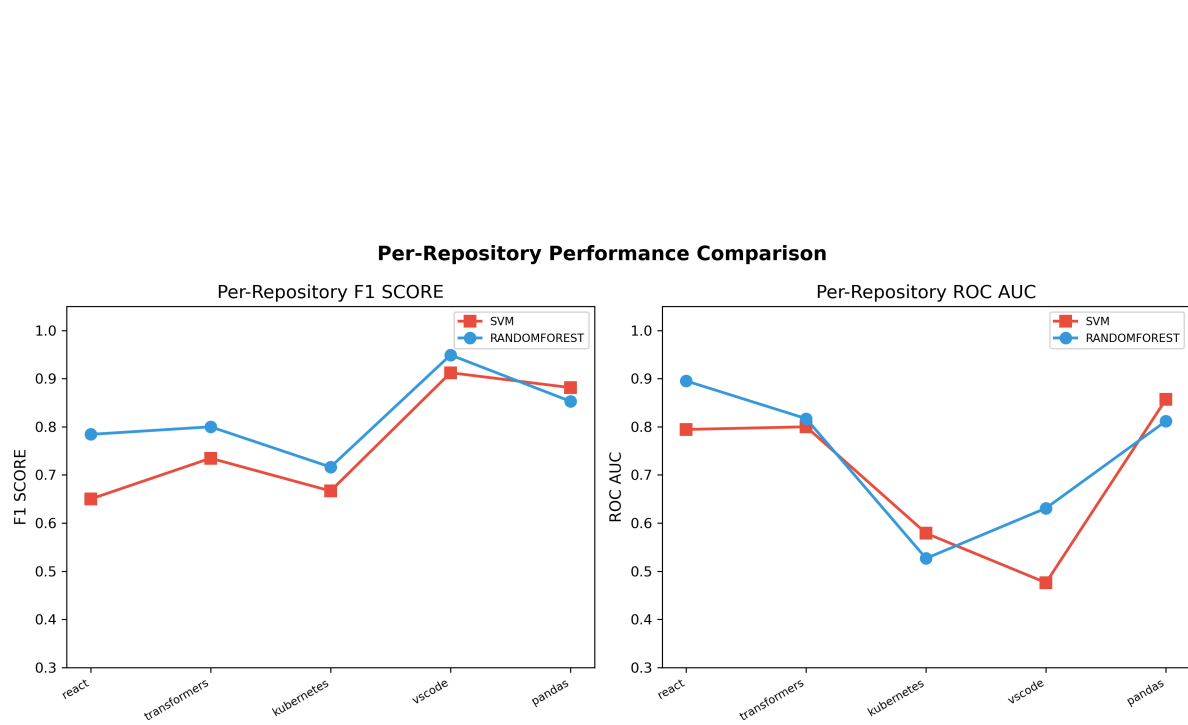


图 9: 分仓库 F1 与 ROC-AUC 对比

四、实验中遇到的问题总结

要求：主要阐述实验过程中遇到的问题如何解决的。

4.1 为什么需要构建 AST 和 CFG 等代码中间表示？

原始代码文本对机器学习模型而言是高维稀疏的，直接使用字符或词级别的特征难以捕捉程序的结构语义。AST 将代码转换为树形结构，保留了语法层次关系（如函数体嵌套在函数定义内）；CFG 将代码的执行路径建模为图结构，揭示了分支、循环和异常处理等控制流逻辑。这些中间表示将”文本”转换为”结构”，使模型能感知代码的复杂度和组织方式，从而提高预测能力。消融实验也验证了这一点：加入 AST 后 F1 从 0.8094 提升至 0.8227。

4.2 为什么传统机器学习需要人工设计特征？

传统机器学习模型（SVM、Random Forest）接受固定长度的数值向量作为输入，本身不具备从原始文本或代码中自动学习表示的能力。因此需要人工将领域知识编码为特征，例如”修改了多少文件””AST 有多少层嵌套””描述有多少个词”。这与深度学习和大语言模型形成对比——后者可以通过神经网络自动从原始输入中学习表示，但代价是更高的训练成本、更大的参数量和更低的解释性。

4.3 SVM 和 Random Forest 分别适用于哪些应用场景？

SVM: 适用于中小规模数据集、特征维度较高的场景。通过核函数（如 RBF）可以处理非线性分类，对特征缩放敏感。SVM 的决策边界由支持向量决定，模型相对简洁。本实验中 SVM 的 Precision (0.8031) 高于 RF (0.7455)，适合对假阳性敏感的场景（如不希望误报一个 PR 会合并）。

Random Forest: 适用于特征间存在复杂非线性关系的场景，对异常值和噪声更鲁棒，不需要特征缩放。RF 天然支持特征重要性评估和并行训练。本实验中 RF 的 Recall (0.9111) 和整体 F1/AUC 均优于 SVM，更适合对召回率敏感的场景（如希望尽可能找出会合并的 PR）。

4.4 哪类特征对 Merge Prediction 贡献最大？为什么？

实验结果显示，**文本特征**（PR 描述长度、Commit Message 长度）是最重要的预测因子（Top-5 中占 3 席），其次是**代码修改规模特征**（删除行数、新增行数）。

原因分析：

- 高质量的 PR 描述和 Commit Message 反映了贡献者的认真程度和沟通能力，这与开源社区的代码审查文化高度一致——清晰的描述降低了审查者的理解成本，提高了合并意愿；
- 修改规模小、聚焦的 PR 更容易被审查者完整阅读和理解，审查周期短，合并概率高；
- AST/CFG 结构特征的贡献相对有限，主要是因为 patch 片段缺少完整的函数和文件上下文，结构特征的信息量天然受限。

4.5 与实验一相比，本实验的数据预处理增加了哪些步骤？

实验一的数据预处理主要包括 API 数据采集、特征提取、统计分析。实验二在此基础上增加了：

1. **数据筛选**：过滤 AI 生成代码 PR (97 条)，删除异常样本 (6 条 `num_changed_files ≤ 0`)，保留 1397 条人类代码 PR；
2. **数据集划分**：按 `repo + is_merged` 分层划分为 train/val/test (70/15/15)，输出稳定的 splits.csv；
3. **Patch 级代码提取**：从 raw JSON 的 Unified Diff 中提取新增代码行（约 225 万行），按文件类型分类；
4. **AST/CFG 构建**：对代码文件进行 tree-sitter AST 解析和简化 CFG 构建，生成 23 个结构特征；
5. **特征工程**：构建 61 维特征向量，包括派生特征（code_churn、per_file 指标）、文件类型分布、语言标记等，并进行 winsorize 截断和 StandardScaler 标准化；
6. **信息泄漏控制**：主动排除审查决策、审阅者数量、标签等后验信息，仅使用 PR 提交时可获得的信息。

4.6 实验中遇到的技术问题

(1) **tree-sitter API 版本适配**。tree-sitter 0.26 的 Python API 与旧版本不同：Parser 构造需直接传入 Language 对象（而非先构造再 set_language），且 tree-sitter-typescript 的语言函数命名为 language_typescript() 和 language_tsx() 而非统一的 language()。通过查阅官方文档和运行时反射（getattr）解决。

(2) **patch_index.json 文件过大**。初始方案存储了每个文件的新增代码行列表，JSON 缩进格式下文件达 122 MB。优化方案为：将代码行列表合并为单个字符串（用 \n 连接），并使用紧凑 JSON（无缩进）序列化，最终压缩至 89.5 MB。

(3) **特征维度的差异导致评估阶段报错**。可视化 ROC 曲线时，主实验模型（61 维特征）与上界实验模型（71 维特征）的输入维度不一致，若统一使用主实验特征文件加载会导致预测失败。解决方式是为每个实验加载对应的特征文件。

(4) **Kubernetes 仓库性能异常偏低**。测试集上 Kubernetes 的 AUC 仅 0.5267，接近随机。排查发现 Kubernetes 的审查流程采用 SIG 机制和多阶段审查，PR 合并决策与代码修改特征的关联模式与其他仓库差异较大，属于仓库特异性问题，非数据泄漏或实现错误。

(5) **Scikit-learn 版本兼容性**。ConfusionMatrixDisplay.from_predictions() 的参数名在不同版本中为 display_labels（非 display_names），需根据实际版本调整。

五、实验体会

通过本实验，我对传统机器学习在代码审查任务中的应用有了系统的实践体验。从数据筛选到特征工程再到模型训练与评估，完整地走通了“原始数据 → 特征向量 → 分类模型”的标准流水线。

主要收获：

(1) **信息泄漏是实验设计中最需要警惕的问题**。主实验与上界实验的对比清晰地表明：如果使用了审查结果特征，模型看似表现优异（AUC 0.95），但这是建立在后验信息上的“虚假繁荣”，实际部署时完全不可用。这一教训对后续所有实验（包括深度学习和 LLM 实验）都有重要指导意义。

(2) **传统机器学习的重要价值在于可解释性**。Random Forest 的特征重要性分析直观地告诉我们：PR 描述的质量和修改规模是决定合并与否的关键因素。这为后续实验六（Prompt 优化）和实验七（VSCode 插件）直接提供了设计方向——可以提示开发者完善 PR 描述、控制修改规模。

(3) **Patch 级别的代码分析有其固有局限**。AST/CFG 特征的边际贡献有限，不是特征工程的问题，而是数据本身的不完整性导致的——patch 只包含局部代码行，缺少完整文件上下文。这一发现为后续使用 Code2Vec、CodeBERT 等表示学习方法提供了有力的动机：深度模型或许能更好地从局部补丁中捕捉代码语义。

(4) **工程实践能力的提升**。从 tree-sitter 多语言配置到 GridSearchCV 超参数搜索，从 patch_index 的大文件优化到分仓库性能分析，本实验涵盖了数据科学项目中的多个实际挑战。

本实验产出的划分文件（splits.csv）、特征矩阵（features_main.csv）和基线模型（SVM/RF）将作为实验三至七的比较基准，确保传统 ML、深度学习、LLM 方法在同一测试集上可公平对比。

指导教师评语：

日期: